TP1
Statistiques descriptives univariées

Analyse du jeu de données Iris

Étudiants : CIPRIANI – VILLENEUVE
Date : 7 septembre 2026

Table des matières

1 Analyse préalable **2**
1.1 Chargement des données – Question 2.1 2
1.2 Nombre d'individus statistiques – Question 2.2 3
1.3 Variables qualitatives – Question 2.3 3
1.4 Variables quantitatives – Question 2.4 3

2 Étude de la variable `species` **4**
2.1 Effectifs – Question 2.5 4
2.2 Représentations graphiques – Question 2.6 4

3 Étude de la variable `petalLength` **5**
3.1 Histogrammes – Question 2.7 5
3.1.1 Histogramme avec 5 intervalles 6
3.1.2 Histogramme avec 10 intervalles 7
3.1.3 Histogramme avec 15 intervalles 8
3.1.4 Histogramme avec 20 intervalles 9
3.2 Analyse de l'influence du nombre d'intervalles – Question 2.8 9
3.2.1 Choix automatique du nombre d'intervalles 10
3.3 Histogramme cumulatif 10
3.4 Boîte à moustaches – Question 2.9 11

4 Résumés numériques de `petalLength` **12**
4.1 Calcul des indicateurs – Question 2.10 12

5 ANNEXES **12**
5.1 Programme complet 12

1 Analyse préalable

1.1 Chargement des données – Question 2.1

Le fichier contenant les données est chargé à l'aide de la bibliothèque `NumPy`.
Le code utilisé est le suivant :

```
from pprint import pprint
import numpy as np

data = np.genfromtxt(
    "iris.csv",
    delimiter=",",
    names=True,
    dtype=None,
    encoding="utf-8"
)

sepallength = data["sepallength"]
sepalwidth = data["sepalwidth"]
petallength = data["petallength"]
petalwidth = data["petalwidth"]
species = data["class"]

speciesname = np.unique(species)
```

La fonction `np.genfromtxt()` permet de lire le fichier CSV et d'en extraire les données grâce à un délimiteur de données (ici le ",").
L'instruction :

```
speciesname = np.unique(species)
```

permet quant à elle de récupérer toutes les valeurs "unique" prises par la variable `species` :

```
Species : ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

Il est possible de faire un parallèle avec la fonction python `Counter()` qui compte et a pour sortie un dictionnaire avec le nombre de mots uniques :

```
liste = [
    "Iris-setosa",
    "Iris-setosa",
    "Iris-setosa",
    "Iris-versicolor",
    "Iris-versicolor",
    "Iris-virginica"
]
compteur = Counter(liste)
```

La sortie est la suivante :

```
Counter({
    "Iris-setosa": 3,
    "Iris-versicolor": 2,
    "Iris-virginica": 1
})
```

Dans le jeu de données étudié, celles-ci sont :

— Iris-setosa ;

— Iris-versicolor ;

— Iris-virginica.

1.2 Nombre d'individus statistiques – Question 2.2

Le nombre d'individus est déterminé à partir du nombre de lignes présentes dans le tableau :

```
nb_individu = len(data)

print(
    f"nombre d'individus statistiques : {nb_individu}"
)
```

On obtient :

$$\boxed{n = 150}$$

Le jeu de données contient donc **150 individus statistiques**.

1.3 Variables qualitatives – Question 2.3

Le jeu de données contient une variable qualitative :

`class`

également appelée `species` dans notre programme.
Cette variable peut prendre trois types comme présenté précédemment :

```
Species : ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

Cette variable est **qualitative nominale**, ici l'ordre ne compte pas (donc elle ne peut pas être ordinale).

1.4 Variables quantitatives – Question 2.4

Les variables quantitatives sont :

Variable	Signification
`sepallength`	Longueur du sépale
`sepalwidth`	Largeur du sépale
`petallength`	Longueur du pétale
`petalwidth`	Largeur du pétale

Ces quatre variables représentent des mesures de "distance" (longueur et largeur plus précisément).
Elles sont donc considérées comme des **variables quantitatives continues**.

2 Étude de la variable species

2.1 Effectifs – Question 2.5

Pour déterminer l'effectif associé à chacune des espèces, le programme suivant est utilisé :

```
dico_data_effectives = dict()

for type in speciesname:
    dico_data_effectives[type] = np.sum(
        species == type
    )

pprint(dico_data_effectives)
```

C'est le principe du dictionnaire qui a été utilisé, car plus simple à manipuler puisqu'il associe un label avec des données. En sortie, il est possible d'observer les résultats suivants :

```
{   np.str_('Iris-setosa'): np.int64(50),
    np.str_('Iris-versicolor'): np.int64(50),
    np.str_('Iris-virginica'): np.int64(50)}
```

Il est donc possible d'établir le tableau suivant :

Espèce	Effectif	Fréquence
Iris-setosa	50	33,3 %
Iris-versicolor	50	33,3 %
Iris-virginica	50	33,3 %
Total	150	100 %

Les trois espèces possèdent donc exactement le même effectif.
Le jeu de données est ainsi parfaitement équilibré par rapport à la variable `species`.

2.2 Représentations graphiques – Question 2.6

Deux représentations graphiques ont été utilisées pour visualiser la répartition des espèces :

- un diagramme en secteurs ;
- un histogramme.

Le code utilisé est :

```
import matplotlib.pyplot as plt

labels = dico_data_effectives.keys()
sizes = dico_data_effectives.values()

fig, (ax1, ax2) = plt.subplots(
    1, 2, figsize=(10, 4)
)

```

```
ax1.pie(
    sizes,
    labels=labels
)

ax2.hist(
    species,
    bins=3,
    edgecolor="black",
    label=labels
)

plt.show()
```

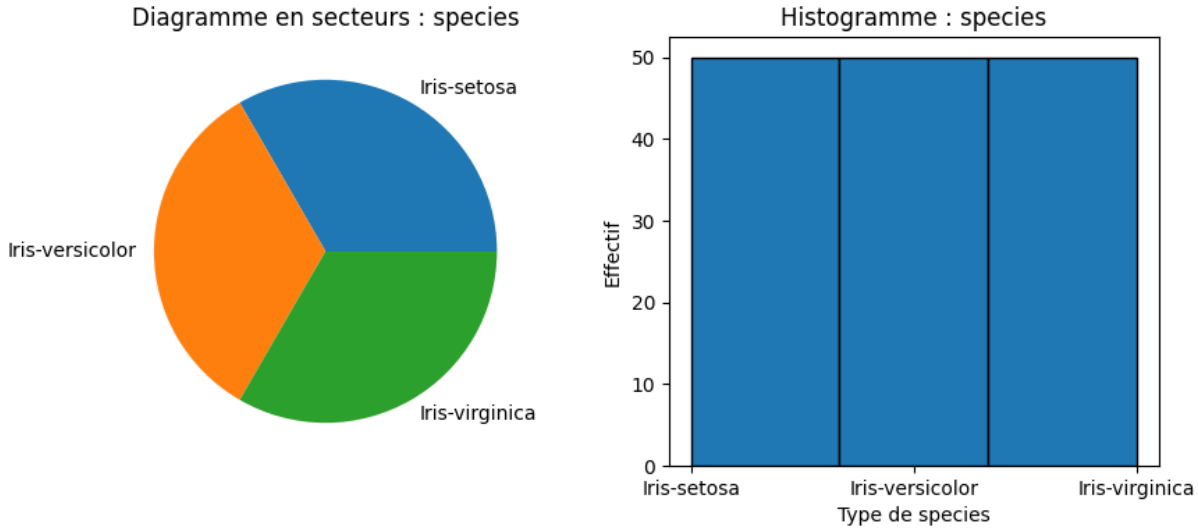


FIGURE 1 – Répartition des individus selon l'espèce.

Le diagramme en secteurs montre une répartition parfaite entre les différentes `species`.

L'histogramme confirme les résultats précédents puisque les trois barres possèdent une hauteur identique de 50 individus.

3 Étude de la variable `petalLength`

3.1 Histogrammes – Question 2.7

L'histogramme permet de représenter la distribution d'une variable quantitative continue en regroupant les observations dans différents intervalles.

Le code général utilisé est :

```
ax1.hist(
    petallength,
    bins=intervalle,
    edgecolor="black"
)
```

Un histogramme cumulatif est également réalisé avec :

```
ax2.hist(
    petallength,
    bins=intervalle,
    cumulative=True,
    edgecolor="black"
)
```

Pour étudier l'influence du nombre d'intervalles sur la représentation, quatre valeurs ont été testées :

$$5, \qquad 10, \qquad 15, \qquad 20.$$

3.1.1 Histogramme avec 5 intervalles

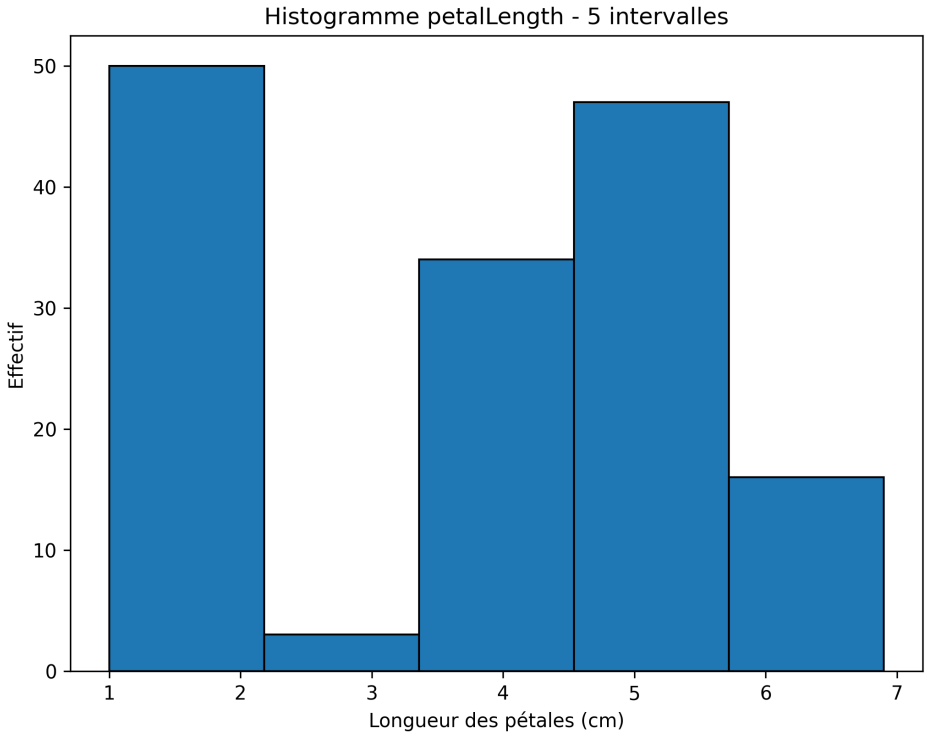


FIGURE 2 – Histogramme de `petalLength` avec 5 intervalles.

Avec seulement cinq intervalles, la représentation est relativement grossière.
On distingue néanmoins deux grandes zones :

— un premier groupe autour de 1 à 2 cm ;

— un second groupe beaucoup plus étendu, approximativement entre 3 et 7 cm.

Une zone de très faible effectif apparaît entre ces deux groupes.

3.1.2 Histogramme avec 10 intervalles

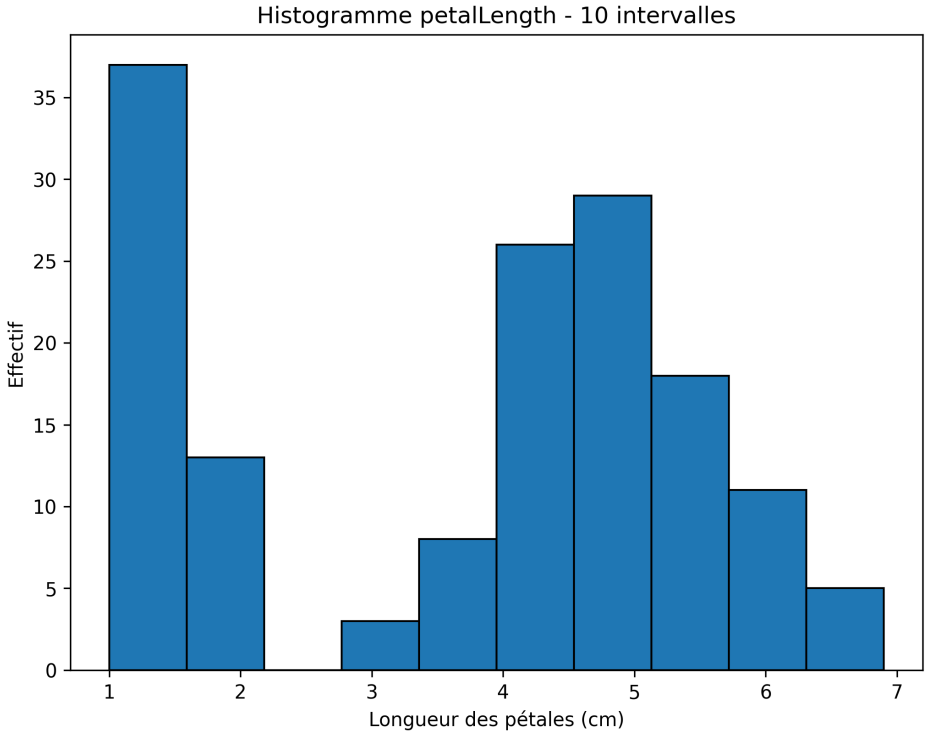


FIGURE 3 – Histogramme de `petalLength` avec 10 intervalles.

Avec dix intervalles, la structure de la distribution devient plus précise.

Le premier groupe est fortement concentré autour des petites longueurs de pétales.

Le deuxième groupe est plus étalé et possède son maximum d'effectif aux alentours de 4 à 5 cm.

La séparation entre le premier groupe et le reste des observations apparaît également plus clairement.

3.1.3 Histogramme avec 15 intervalles

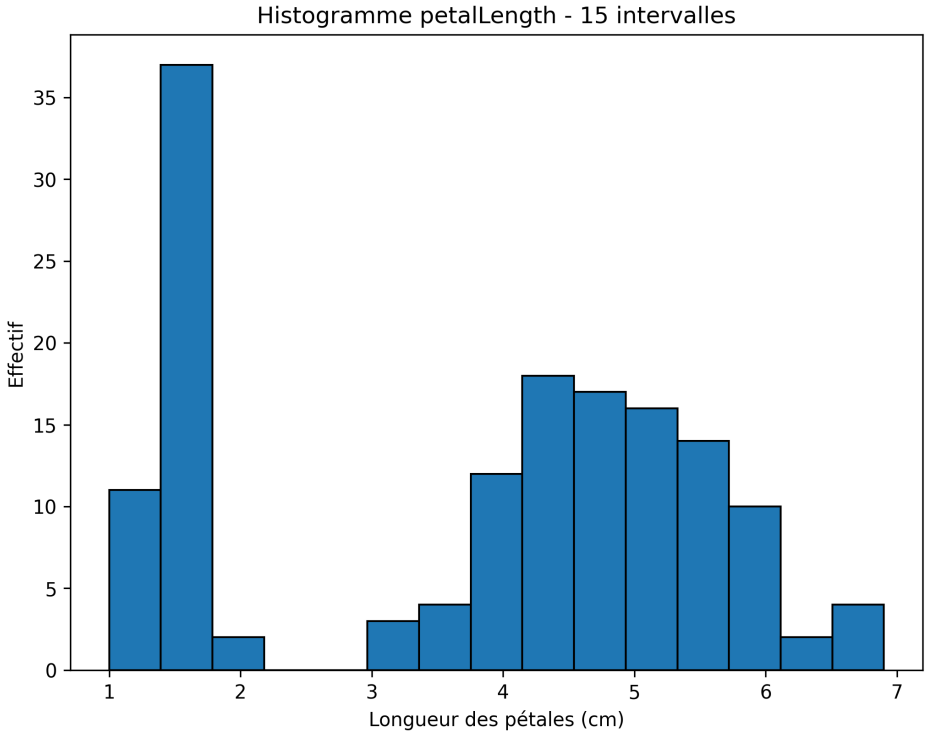


FIGURE 4 – Histogramme de `petalLength` avec 15 intervalles.

Avec quinze intervalles, davantage de détails apparaissent dans la distribution.

La concentration du premier groupe autour de 1,5 cm devient particulièrement visible.

Pour les longueurs supérieures à environ 3 cm, la distribution est plus étendue et les variations locales des effectifs deviennent plus faciles à observer.

3.1.4 Histogramme avec 20 intervalles

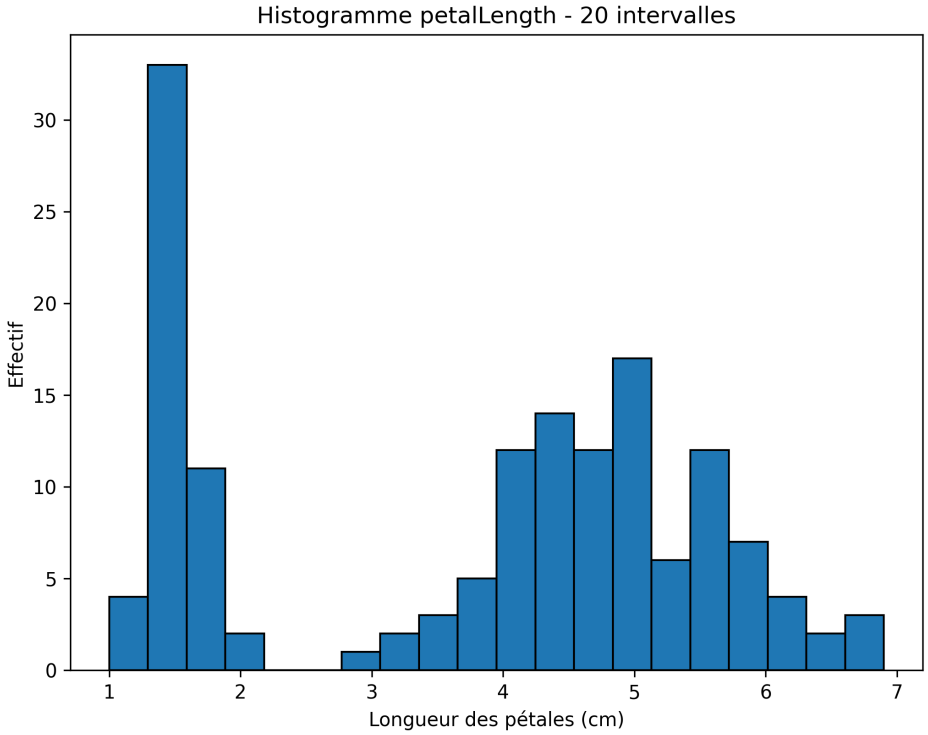


FIGURE 5 – Histogramme de `petalLength` avec 20 intervalles.

Avec vingt intervalles, la résolution de l'histogramme est encore augmentée.

Les fluctuations locales des données deviennent davantage visibles. Cependant, la représentation devient également moins synthétique : chaque intervalle contient moins d'observations et la forme générale de la distribution est donc plus irrégulière.

3.2 Analyse de l'influence du nombre d'intervalles – Question 2.8

La comparaison des quatre histogrammes met en évidence l'importance du choix du nombre d'intervalles.

Avec un faible nombre d'intervalles, par exemple cinq, la représentation est très grossière et ne permet pas d'identifier une partie des variations internes.

Lorsque le nombre d'intervalles augmente, la largeur de chaque classe diminue. La représentation devient donc plus détaillée.

Cependant, un nombre trop important d'intervalles peut donner une impression de forte irrégularité, car chaque classe contient alors un nombre plus faible d'observations.

Malgré ces différences de représentation, une caractéristique reste présente quel que soit le nombre d'intervalles : la distribution de `petalLength` n'est pas unimodale.

On observe une forte concentration des observations pour des longueurs comprises approximativement entre 1 et 2 cm, puis une seconde zone beaucoup plus large entre environ 3 et 7 cm.

La zone comprise approximativement entre 2 et 3 cm contient très peu d'observations.

Cette structure montre que la population totale n'est pas homogène du point de vue de la longueur des pétales.

Le choix du nombre d'intervalles ne modifie donc pas les données elles-mêmes, mais modifie la manière dont leur structure est rendue visible.

Dans le cas présent, une valeur intermédiaire, par exemple entre 10 et 15 intervalles, constitue un compromis intéressant entre lisibilité et niveau de détail.

3.2.1 Choix automatique du nombre d'intervalles

Le programme contient également deux estimations du nombre d'intervalles :

```
n = len(petallength)

intervalle_Sturge = int(
    np.ceil(1 + 3.3 * np.log10(n))
)

intervalle_yule = int(
    np.ceil(2.5 * n**0.25)
)
```

Pour :

$$n = 150,$$

la règle de Sturges donne approximativement :

$$k = \lceil 1 + 3,3 \log_{10}(150) \rceil$$

soit :

$$\boxed{k = 9}.$$

La formule utilisée dans le programme pour Yule donne :

$$k = \lceil 2,5 \times 150^{1/4} \rceil$$

soit également environ :

$$\boxed{k = 9}.$$

Ces valeurs sont proches du choix de dix intervalles testé précédemment. Il est aussi à noter qu'il existe un paramètre modifiable `Bins` qui peut être initialisé à "auto" pour les intervalles. Lors de la simulation, celui-ci renvoie 9 intervalles sur l'histogramme.

3.3 Histogramme cumulatif

L'histogramme cumulatif représente non plus l'effectif de chaque intervalle pris indépendamment, mais l'effectif cumulé depuis la plus petite valeur.

Ainsi, la hauteur de la dernière barre correspond nécessairement au nombre total d'observations :

150

Cette représentation permet notamment d'observer la proportion d'individus dont la longueur de pétale est inférieure à une valeur donnée.

3.4 Boîte à moustaches – Question 2.9

La boîte à moustaches est construite avec :

```
ax3.boxplot(petallength)
ax3.set_title(
    "Boite a moustache : petallength"
)
```

L'histogramme, l'histogramme cumulatif et la boîte à moustaches sont présentés sur la figure suivante.

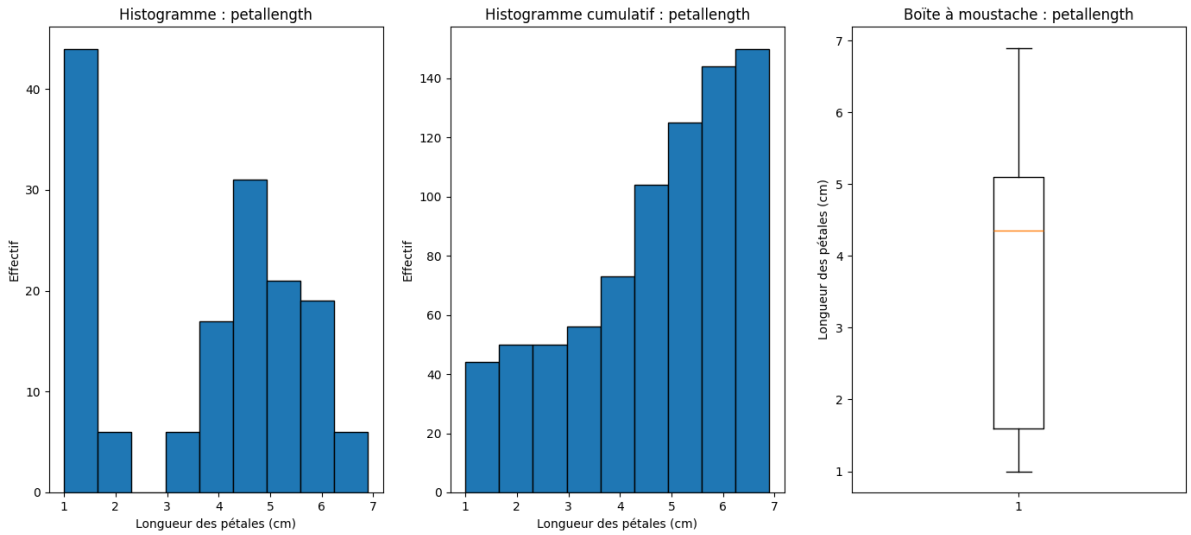


FIGURE 6 – Histogramme, histogramme cumulatif et boîte à moustaches de la variable `petalLength`.

Une boîte à moustaches est principalement constituée des éléments suivants :

- le premier quartile Q_1, correspondant au bord inférieur de la boîte ;
- la médiane Q_2, représentée par le trait situé à l'intérieur de la boîte ;
- le troisième quartile Q_3, correspondant au bord supérieur de la boîte ;
- l'écart interquartile :
$$IQR = Q_3 - Q_1;$$
- les moustaches, qui s'étendent vers les valeurs extrêmes suivant la convention utilisée par le logiciel ;
- les éventuelles valeurs aberrantes, qui peuvent être représentées individuellement au-delà des moustaches.

Dans notre cas :

$$Q_1 = 1,6 \text{ cm}$$

et :

$$Q_3 = 5,1 \text{ cm}.$$

L'écart interquartile vaut donc :

$$IQR = 5,1 - 1,6$$

soit :

$$\boxed{IQR = 3,5 \text{ cm}}.$$

La médiane, située à :

$$\boxed{4,35 \text{ cm}},$$

est également visible à l'intérieur de la boîte.

La boîte est relativement étendue, ce qui traduit une dispersion importante des longueurs de pétales lorsque les trois espèces sont considérées simultanément.

4 Résumés numériques de `petalLength`

4.1 Calcul des indicateurs – Question 2.10

Les différents indicateurs statistiques sont calculés avec NumPy.

Le programme utilisé est :

```
mean_petallength = np.mean(petallength)
median_petallength = np.median(petallength)

ecartType_petallength = np.std(petallength)
variance_petallength = np.var(petallength)

quartile_petallength = np.quantile(
    petallength,
    q=[0.25, 0.75]
)
```

Les résultats obtenus par le programme sont :

Indicateur	**Valeur**
Moyenne	$3,7587$ cm
Médiane	$4,35$ cm
Écart-type (`ddof=0`)	$1,7585$ cm
Variance (`ddof=0`)	$3,0924$ cm^2
Premier quartile Q_1	$1,6$ cm
Troisième quartile Q_3	$5,1$ cm
Écart interquartile	$3,5$ cm

5 ANNEXES

5.1 Programme complet

```python
# CIPRIANI VILLENEUVE

# Question 2.1:
from pprint import pprint
import numpy as np
# Lire le fichier CSV avec numpy (assume un header)
data = np.genfromtxt("iris.csv", delimiter=",",
names=True, dtype=None, encoding="utf-8")
# Colonnes
sepallength = data["sepallength"]
sepalwidth = data["sepalwidth"]
petallength = data["petallength"]
petalwidth = data["petalwidth"]
species = data["class"]
speciesname = np.unique(species)

print(f"Species : {speciesname}")

save_hist_variants: bool = True
status : bool = True #Permet de skip les graphiques

# 2.1 Analyse pr alable

# Question 2.2:
nb_individu : int  = len(data)
print(f"nombre  d individus  statistiques : {nb_individu}")

# 2.2 Etude de la variable species

# Question 2.5:
dico_data_effectives = dict()
for type in speciesname:
    # dico_data_effectives[type] = {}
    dico_data_effectives[type]= np.sum(species == type)

pprint(dico_data_effectives, indent=4, width=80)

# Question 2.6:
import matplotlib.pyplot as plt

labels = dico_data_effectives.keys()
sizes = dico_data_effectives.values()
print(sizes)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.pie(sizes, labels=labels)
ax1.set_title("Diagramme en secteurs : species")

ax2.hist(species, bins=3, edgecolor='black',label=labels)
ax2.set_title("Histogramme : species")
ax2.set_xlabel("Type de species")
```

```
ax2.set_ylabel("Effectif")
# plt.savefig("graphique_2_2.png")
plt.show(block = status)

# 2.3  tude  de la variable petalLength

# Question 2.7:
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(17, 7))

# intervalle = 10
intervalle = "auto"
n  = len(petallength)
intervalle_Sturge = int(np.ceil(1 + 3.3 * np.log10(n)))
intervalle_yule = int(np.ceil(2.5 * n**0.25))
print(f"Intervalle Sturge : {intervalle_Sturge}\nIntervalle Yule :
    {intervalle_yule}")

plt.title(f"Etude de la variable petalLength avec intervalle : {
    intervalle}")
# ax1.hist(petallength, bins=intervalle_Sturge, edgecolor='black')
ax1.hist(petallength, bins=intervalle, edgecolor='black')
ax1.set_title("Histogramme : petallength")
ax1.set_xlabel("Longueur des petales (cm)")
ax1.set_ylabel("Effectif")
# ax2.hist(petallength, bins=intervalle_Sturge,cumulative = True,
    edgecolor='black')
ax2.hist(petallength, bins=intervalle,cumulative = True, edgecolor=
    'black')
ax2.set_title("Histogramme cumulatif : petallength")
ax2.set_xlabel("Longueur des petales (cm)")
ax2.set_ylabel("Effectif")

#Dedie au rapport
if save_hist_variants:

    liste_intervalles = [5, 10, 15, 20]

    for nb_intervalles in liste_intervalles:

        fig_hist, ax_hist = plt.subplots(figsize=(8, 6))

        ax_hist.hist(
            petallength,
            bins=nb_intervalles,
            edgecolor="black"
        )

        ax_hist.set_title(
            f"Histogramme petalLength - {nb_intervalles}
    intervalles"
        )
```

```
98          ax_hist.set_xlabel("Longueur des p tales (cm)")
99          ax_hist.set_ylabel("Effectif")
100
101         nom_fichier = f"histogramme_petallength_{nb_intervalles}
        _intervalles.png"
102
103         fig_hist.savefig(nom_fichier, dpi=300, bbox_inches="tight")
104
105         plt.close(fig_hist)
106
107 # Question 2.8:
108
109 # Question 2.9:
110 ax3.boxplot(petallength)
111 ax3.set_title("Boite    moustache : petallength")
112 ax3.set_ylabel("Longueur des petales (cm)")
113 name = "graphique_2_3_interv_"+ str(intervalle) +".png"
114 # plt.savefig("graphique_2_3_interv_.png")
115 plt.show(block = status)
116
117
118 # Deuxieme approche : resumes numeriques
119 # Question 2.10:
120 mean_petallength = np.mean(petallength)
121 median_petallength = np.median(petallength)
122 print(f"Mean : {mean_petallength}\nMedian : {median_petallength}")
123
124 ecartType_petallength = np.std(petallength)
125 variance_petallength = np.var(petallength)
126 quartile_petallength = np.quantile(petallength, q=[0.25, 0.75])
127 print(f"Ecart_type : {ecartType_petallength}\nVariance : {
        variance_petallength}\nQuartile : {quartile_petallength}")
128
129 # Mean : 3.758666666666666
130 # Median : 4.35
131 # Ecart_type : 1.7585291834055212
132 # Variance : 3.092424888888889
133 # Quartile : [1.6 5.1]
```